

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	K.HEMASREE	Reg. No.:	192411156
Section / Batch:	SLOT C	Date:	27/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I, K.HEMA(192411156)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AIM

To develop a **backtracking-based solution for Resource-Constrained Project Scheduling (RCPSP)** that schedules project tasks while satisfying **resource limitations, task dependencies (precedence constraints), and a given project deadline**.

ALGORITHM

1. **Start** the program.
2. Define each task with:
 - Task ID
 - Duration
 - Resource requirement
 - Predecessor tasks
3. Initialize:
 - An empty schedule.
 - Resource usage for every time unit up to the deadline.
4. Select the **next unscheduled task**.
5. Determine the **earliest possible start time** based on the completion times of all its predecessor tasks.
6. Try possible start times from the earliest start time up to the deadline.
7. For each possible start time:
 - Check whether the task can finish before the deadline.
 - Check whether sufficient resources are available during its entire execution.
8. If all constraints are satisfied:
 - Assign the task to that time interval.
 - Update the resource usage.
 - Store the task's start time and finish time.
9. Recursively schedule the remaining tasks.
10. If scheduling fails for the current choice:
 - **Backtrack**.
 - Remove the task from the schedule.
 - Restore the resource availability.
11. Continue until:
 - All tasks are successfully scheduled → **Valid Schedule Found**.
 - No possible arrangement exists → **No Valid Schedule Found**.
12. **Stop**.

COS AT1-Constraint-Based Problem Solving

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Resource-Constrained Project
Scheduling (NP-Hard Problem)
100 marks

1/1 answered

Q1 Resource-Constrained Project Scheduling (NP-Hard Problem)

100 marks

Schedule project tasks with limited resources and dependencies. Clearly define constraints such as resource limits and precedence relations. Analyze how constraints affect scheduling feasibility. Develop a backtracking-based solution. Apply pruning techniques to eliminate invalid schedules. Justify efficiency and discuss NP-Hard complexity.

Your Code

```
1 class Task:
2     def __init__(self, id, duration, resource_req, predecessors):
3         self.id = id
4         self.duration = duration
5         self.resource_req = resource_req
6         self.predecessors = predecessors
7
8     def schedule_tasks(tasks, max_resource, deadline):
9         schedule = {}
10        resource_timeline = [0] * (deadline + 1)
11
12        def backtrack(task_idx):
13            if task_idx == len(tasks):
14                return True
15
16            current_task = tasks[task_idx]
17
18            # Calculate earliest start time based on predecessors
19            earliest_start = 0
20            for pred in current_task.predecessors:
21                if pred in schedule:
22                    # Find the predecessor task to look up its duration
23                    pred_task = next(t for t in tasks if t.id == pred)
24                    finish = schedule[pred] + pred_task.duration
```

Input

stdin input

Output

```
Valid Schedule Found (Start
Times):
Task 1: Starts at t=0, Finishes
at t=3
Task 2: Starts at t=3, Finishes
at t=6
```